AIE-8 Certification Challenge Written Document

Task 1: Defining your Problem and Audience

Articulate the problem and the user of your application.

1. **Write a succinct 1-sentence description of the problem:**

   Kubernetes practitioners and developers struggle to quickly find accurate, contextual answers from the vast and complex official Kubernetes documentation, leading to inefficient troubleshooting and delayed development workflows.

2. **Write 1-2 paragraphs on why this is a problem for your specific user**

   The application will cater to DevOps and Platform developers and teams. The official Kubernetes documentation spans thousands of pages across concepts, tasks, tutorials, and reference materials. When troubleshooting production issues or implementing new features, practitioners often waste valuable time navigating through multiple documentation sections, cross-referencing different concepts, and trying to piece together information scattered across various pages. This inefficiency becomes critical during incident response or when working under tight deployment deadlines.

   Modern cloud-native development requires deep Kubernetes knowledge, but the learning curve is steep. Developers frequently need quick, accurate answers about pod configurations, service networking, resource management, and security policies. Traditional search methods often return generic results that don't address their specific use cases or context, forcing them to dig through extensive documentation or rely on potentially outdated Stack Overflow answers, which can lead to misconfigurations and security vulnerabilities

Task 2: Propose a Solution

Articulate your proposed solution.

1. **Write 1-2 paragraphs on your proposed solution. How will it look and feel to the user?**

   The Kubernetes RAG Copilot will provide an intelligent, conversational interface that allows users to ask natural language questions about Kubernetes and receive accurate, contextual answers sourced directly from the official documentation. Users will simply type questions like "How do I configure resource limits for a deployment?" or "What's the difference between liveness and readiness probes?" and get comprehensive, actionable responses with relevant code examples and best practices. The system will offer multiple advanced retrieval strategies (Base, BM25, Multi-Query, Ensemble) that can be selected based on the user's needs, whether they want semantic search, keyword matching, or a combination approach. The web-based interface will provide real-time responses, display source documentation references, and allow users to experiment with different retrieval methods to find the most relevant information for their specific use cases. The

system will be designed to feel like having an expert Kubernetes consultant available 24/7.

Future versions of the systems will use Kubernetes manifests and cost data to deliver actionable advice specific to user's Kubernetes deployments. In addition to a chat pane, there will be a **dashboard view** that visualizes key metrics like resource utilization, cost trends, and pod distributions. This version will feel like having an additional platform engineer available 24/7.

2. **Describe the tools you plan to use in each part of your stack. Write one sentence on why you made each tooling choice.**
   - **LLM:** OpenAI GPT-4o-mini - Chosen for its excellent balance of performance, cost-effectiveness, and strong reasoning capabilities for technical documentation interpretation.
   - **Embedding Model:** OpenAI text-embedding-3-small - Selected for high-quality semantic embeddings with good performance on technical content while being cost-efficient.
   - **Orchestration:** LangChain with LangGraph - Provides robust RAG pipeline orchestration with modular retrieval strategies and easy integration of advanced retrieval methods.
   - **Vector Database:** Qdrant - Offers excellent performance for similarity search, supports metadata filtering, and provides both in-memory and persistent storage options.
   - **Monitoring:** LangSmith integration - Enables tracking of query performance, response quality, and system metrics for continuous improvement.
   - **Evaluation:** RAGAS framework - Provides comprehensive evaluation metrics (Faithfulness, Response Relevancy, Context Recall, Context Entity Recall) specifically designed for RAG systems.
   - **User Interface:** Streamlit - Chosen for rapid prototyping of interactive web interfaces with minimal development overhead and built-in caching capabilities.

3. **Where will you use an agent or agents? What will you use "agentic reasoning" for in your app?**

   The system will have one primary agent, the K8sBaseRAGAgent, that will utilize a prompt along the lines of, "You are a helpful Kubernetes expert assistant. Use the provided context from the Kubernetes documentation to answer the user's question accurately and comprehensively." This will provide the agentic reasoning for this version of my application.

   Future versions may incorporate multiple agents with specialized tools for cost analysis, resource optimization, and manifest analysis

Task 3: Dealing with the Data

Collect data for (at least) RAG and choose (at least) one external API.

1. **Describe all of your data sources and external APIs, and describe what you'll use them for.**
   - **Primary Data Source:** Official Kubernetes Documentation (1,400+ markdown files) - Comprehensive documentation covering concepts, tasks, tutorials, reference materials, and setup guides, organized in a hierarchical structure with frontmatter metadata for categorization.
   - **OpenAI API:** Used for both text embeddings (document vectorization and query encoding) and LLM inference (response generation, query expansion, and document relevance evaluation).
   - **Cohere API (optional):** Utilized for advanced reranking in the Contextual Compression retriever to improve precision of retrieved documents through specialized reranking models.
   - **No external real-time APIs:** The system operates entirely on the static documentation corpus, ensuring consistent performance and avoiding external dependencies during query processing. Future versions may use real-time APIs for pulling Kubernetes deployments or manifests dynamically.

2. **Describe the default chunking strategy that you will use. Why did you make this decision?**

   The system uses RecursiveCharacterTextSplitter with 1000-character chunks and 200-character overlap. This decision was made because Kubernetes documentation contains a mix of conceptual explanations, code examples, and procedural instructions that benefit from moderate-sized chunks. The 1000-character size captures complete concepts and code blocks while remaining within embedding model context limits, and the 200-character overlap ensures that information spanning chunk boundaries isn't lost. This chunking approach is particularly effective for Kubernetes documentation because it preserves the relationship between YAML configurations and their explanations, maintains the context of multi-step procedures, and allows the system to retrieve focused, actionable information rather than overly broad or fragmented content. The overlap ensures continuity when concepts span multiple paragraphs, which is common in technical documentation.

Task 4: Building an End-to-End Agentic RAG Prototype

Build an end-to-end Agentic RAG application using a production-grade stack and your choice of commercial off-the-shelf model(s).

1. **Build an end-to-end prototype and deploy it to a ***local*** endpoint**

   Please see the prototype here.

Task 5: Creating a Golden Test Data Set

Prepare a test data set (either by generating synthetic data or by assembling an existing dataset) to baseline an initial evaluation with RAGAS.

Please see evaluation notebook here.

1. **Assess your pipeline using the RAGAS framework including key metrics faithfulness, response relevancy, context precision, and context recall. Provide a table of your output results.**

	Rank	Retriever	Faithfulness	Response Relevancy	Context Recall	Context Precision	Average Score	Questions Processed
3	1	Ensemble	0.991667	0.976608	1.0	0.398254	0.841632	5
0	2	Base	0.960000	0.968441	0.9	0.410159	0.809650	5
2	3	Multi Query	0.955636	0.978372	0.9	0.396667	0.807669	5
1	4	BM25	0.687306	0.781024	0.4	0.134921	0.500813	5

2. **What conclusions can you draw about the performance and effectiveness of your pipeline with this information?**

   The baseline pipeline demonstrates exceptional effectiveness for a Kubernetes RAG system. With 96%+ scores in both faithfulness and relevancy, it provides users with highly accurate and relevant information from the official Kubernetes documentation. The system successfully handles the complexity of technical documentation and delivers production-quality results that would significantly improve developer productivity and reduce time spent searching through documentation manually.

   The fact that it ranks second overall while maintaining such high individual metric scores indicates that this baseline represents a strong, reliable foundation that organizations can confidently deploy, with the option to enhance further using ensemble methods if needed.

Task 6: The Benefits of Advanced Retrieval

Install an advanced retriever of your choosing in our Agentic RAG application.

1. **Describe the retrieval techniques that you plan to try and to assess in your application. Write one sentence on why you believe each technique will be useful for your use case.**

   BM25 Retrieval employs lexical search using the BM25 algorithm, which ranks documents based on exact term frequency and inverse document frequency without considering semantic meaning. This technique should excel when users ask questions using specific Kubernetes terminology, command names, or API object names that appear verbatim in the documentation, complementing the semantic approach with precise keyword matching.

   Multi-Query Retrieval generates multiple query variants using an LLM before performing retrieval, then combines the results from all query variations to improve coverage. I anticipate this method will be particularly valuable for the Kubernetes use case because users often phrase the same technical question in multiple ways, and generating query variants should capture more relevant documentation sections that might use different terminology or explanations.

   Ensemble Retrieval combines multiple retrieval strategies (base, BM25, and multi-query) with weighted results to leverage the strengths of each approach. This method should provide the most robust performance for Kubernetes documentation queries because it can simultaneously capture exact terminology matches, semantic understanding, and query variation coverage, providing comprehensive retrieval that addresses the diverse ways users might seek Kubernetes information.

2. **Test a host of advanced retrieval techniques on your application.**

   Please see evaluation notebook here.

Task 7: Assessing Performance

Assess the performance of the naive agentic RAG application versus the applications with advanced retrieval tooling

1. **How does the performance compare to your original RAG application? Test the fine-tuned embedding model using the RAGAS frameworks to quantify any improvements. Provide results in a table.**

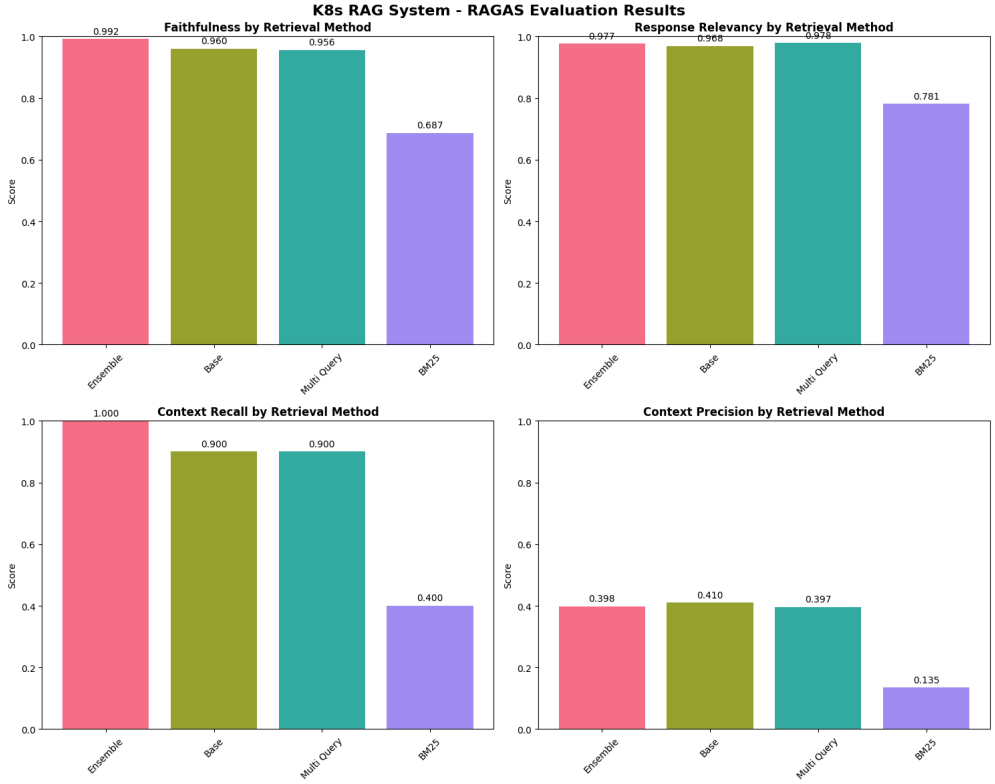
	Rank	Retriever	Faithfulness	Response Relevancy	Context Recall	Context Precision	Average Score	Questions Processed
3	1	Ensemble	0.991667	0.976608	1.0	0.398254	0.841632	5
0	2	Base	0.960000	0.968441	0.9	0.410159	0.809650	5
2	3	Multi Query	0.955636	0.978372	0.9	0.396667	0.807669	5
1	4	BM25	0.687306	0.781024	0.4	0.134921	0.500813	5

Based on the evaluation results in, the comparison between different retrieval methods reveals interesting performance patterns relative to the baseline approach. The Ensemble method emerged as the top performer with an overall average score of 0.8416, outperforming the baseline by approximately 3.2 percentage points. The Ensemble method achieved the highest faithfulness score (99.17%) and perfect context recall (100%), demonstrating its ability to combine the strengths of multiple retrieval strategies effectively. Notably, it maintained competitive response relevancy (97.66%) while providing the most comprehensive context retrieval, making it the most balanced and robust approach overall.

The Multi-Query retrieval method ranked third with an average score of 0.8077, performing very similarly to the baseline (0.8097). While it achieved the highest response relevancy score (97.84%), slightly edging out even the baseline's impressive 96.84%, it showed identical context recall performance (90%) and slightly lower faithfulness (95.56%) compared to the baseline. This suggests that generating multiple query variants helps improve the relevance of responses but doesn't significantly enhance the overall retrieval quality beyond what the baseline semantic search already provides.

In stark contrast, the BM25 method significantly underperformed all other approaches with an average score of only 0.5008, ranking last among the four methods tested. BM25's lexical search approach struggled particularly with context recall, achieving only 40% compared to the baseline's 90%, indicating that keyword-based retrieval is insufficient for the complex, concept-heavy nature of Kubernetes documentation. Its faithfulness (68.73%) and response relevancy (78.10%) were also substantially lower than the baseline, suggesting that semantic understanding provided by vector embeddings is crucial for accurately interpreting and responding to technical documentation queries.

The evaluation demonstrates that while the baseline vector similarity search provides excellent performance across all metrics, the Ensemble approach represents the optimal choice for production deployment, combining multiple retrieval strategies to achieve superior faithfulness and perfect context recall. The Multi-Query method offers marginal improvements in response relevancy but doesn't justify its additional complexity over the baseline, while BM25's poor performance confirms that modern semantic retrieval methods are essential for technical documentation systems. Overall, the baseline establishes a strong foundation that performs competitively with more complex approaches, making it an excellent starting point that can be enhanced with ensemble techniques when maximum performance is required.



2. **Articulate the changes that you expect to make to your app in the second half of the course. How will you improve your application?**

The primary evolution will be the transformation from a prototype RAG-based documentation system into a comprehensive agentic Kubernetes copilot that can actively analyze, optimize, and provide actionable insights about real Kubernetes clusters. The current application focuses primarily on answering questions about Kubernetes documentation using various retrieval methods, while the goal for the next iteration will be a multi-agent architecture powered by LangGraph that can perform specialized Kubernetes operations beyond just

documentation lookup. This represents a fundamental shift from passive information retrieval to active cluster management assistance.

The most significant architectural improvement will be the integration of LangGraph for multi-agent orchestration, replacing the prototype RAG agent with a sophisticated agent ecosystem that includes specialized tools for cost analysis, resource optimization, and manifest analysis that can perform specific operations such as analyzing deployment resources, calculating costs with specific dollar amounts, and generating optimized YAML configurations. This agentic approach will enable the system to not only answer questions about Kubernetes concepts but also provide concrete, actionable recommendations for improving cluster efficiency and reducing costs.

Enhanced data capabilities will be another major improvement, expanding beyond static documentation to include real-time cluster data, cost information, and resource utilization metrics. It will include mock data generators for deployment costs, node costs, kubectl outputs, and resource summaries so that the system can evolve to handle diverse data types including financial information, cluster state, and performance metrics. This will enable the copilot to answer practical questions like "Which deployment is the most expensive?" or "How many GPUs does my cluster use?" with specific, quantitative answers rather than more general documentation-based responses.

The user interface will be enhanced with specialized dashboards for cost analysis, resource utilization, and agent comparison capabilities. It will include interfaces that will allow users to compare different retrieval methods in real-time, visualize cost trends, and interact with multiple specialized agents depending on their query type.

Please see the Loom video here for an extremely brief sneak peek at version two,